

Auditing a degree against the industry it feeds

Candidate: Poojitha Branch: B.Tech CSE Regulation: R22

Target grad year: 2030 Updated: 16 Sept 2026

Status

Eight sections, all evidence are backed.

Section	Status
1 — Current degree	Done
2 — What's being taught	Done — all 8 semesters mapped, R22 vs R25 diff verified
3 — Subject labels	Done — 10 of 10
4 — 2026 field-change reality	Done
5 — Curriculum vs job postings	Done — 3 postings checked
6 — 2030 redesign	Done — credit budget resolved
a/b — AI capability test	Done — 6 subjects tested
Human conversation	Done — 2024 grad, Platform Engineer
7 — What would make me wrong	Done
8 — AI claim checked & found wrong	Done

GitHub repo

Done

Live report + interactive feature . Done

Key findings

- Stats moved into the same semester as ML under R25, breaking a prerequisite gap that R22 kept at three semesters.
-
- DevOps structure and credits are untouched in R25 — the problem is tooling. Syllabus still teaches Jenkins, manual kubectl, Selenium; industry has moved to GitHub Actions, ArgoCD, Terraform, Prometheus+Grafana, Trivy+SonarQube.
-
- The HKAIVERSE brief's claim that R25 cut DevOps from 4 credits to 1 does not hold — checked against the official bulletin and the uploaded R25 PDF, both show it unchanged.
-
- Compiler Design and Formal Languages were merged in R25. Grad interview reframed this from REDUCE to UPDATE — the theory (AST, optimization) still matters, but the delivery should move to modern tooling instead of hand-written parsers.
-
- Both open electives on offer are cross-branch, non-CS subjects — evidence the open elective basket isn't doing anything for CS depth.
-
- Computer Networks flipped from KEEP to UPDATE after the grad described labs still running RIP in Packet Tracer against a real world of VPCs, load balancers, and edge networking.
-
- Git is a double-sourced gap: flagged independently by a job posting and by the grad, who force-pushed to main on day one with no rebase or PR-review training behind them.
-

- Cloud Computing already exists as a professional elective in R22 — the fix is promoting it to core, not adding a new course.

Section 3 — Subject labels

Every core subject, one label, one line of evidence. Filter by label.

ALL 10

KEEP 2

UPDATE 7

REDUCE 1

ADD 0

01 Machine Learning

Stats prerequisite moved into the same semester as ML in R25, breaking sequencing that worked in R22.

UPDATE

02 DevOps

Structure and credits are fine; tooling is stale. Grad confirms: never opened a .github/workflows file before the job.

UPDATE

03 Compiler Design + Formal Languages

Keep AST/optimization concepts, teach via modern tooling (ESLint/Babel-style) instead of hand-written parsers.

UPDATE

04 Artificial Intelligence

Core AI stays classical; 2026 job postings want LLM APIs, RAG, agents.

UPDATE

05 Design & Analysis of Algorithms

Complexity/DP/graph algorithms remain fixed in every 2026 SDE interview loop.

KEEP

06 Operating Systems

Grad confirms: OOMKilled pods and CPU throttling in K8s are literally OS concepts in practice.

KEEP

07 Database Management Systems

RDBMS/SQL-only; 2026 postings need NoSQL and vector DBs for RAG.

UPDATE

08	Computer Networks	UPDATE
	Flipped from KEEP after grad interview: RIP/Packet Tracer labs vs. real VPCs, load balancers, Cloudflare.	
09	Cryptography & Network Security	UPDATE
	Classical crypto only; 2026 postings emphasize cloud security — IAM, zero-trust.	
10	Organizational Behaviour (IV-II)	REDUCE
	R25 already dropped it as core; not listed in any 2026 job posting checked.	

Section 4 — 2026 reality

What the syllabus assumes about each domain, against what's actually true in 2026.

Domain	2022 assumption	2026 reality
AI / ML	Classical ML + basic AI	LLMs, RAG, agentic workflows dominant
DevOps	Jenkins, manual kubectl, Selenium	GitHub Actions, ArgoCD GitOps, Terraform, Trivy
Databases	RDBMS-only (SQL)	Vector DBs standard for RAG apps
Security	Perimeter / crypto-only	Zero-trust, cloud IAM, LLM security
Compilers	Manual parser theory	LLVM/ANTLR/ESLint-style tooling-first practice
Software delivery	Waterfall/Agile theory	AI coding agents default — 90% weekly, 68% daily use (JetBrains 2026 Dev Ecosystem Survey, 15,000+ devs)
Networking	Static TCP/IP / RIP	VPCs, load balancers, Cloudflare-style edge
Stats/ML sequencing	Prerequisite-before-use	R25 broke this — Stats and ML now co-placed

Section 5 — Curriculum vs. industry

Three real 2026 job postings checked against current coverage.

- **JD1** — Glassdoor India, Sept 2026: Azure, Go, Node.js, React, TypeScript/NPM, CI/CD, cloud infra, Kubernetes, Ansible.
- **JD2** — Job Zone Career multi-company drive, Sept 2026: Java, C#, .NET, Python, React, Angular, Spring Boot, Microservices, Kafka, REST APIs, Kubernetes, Azure, Git, CI/CD, IoT.
- **JD3** — AmQuest 2026 skills guide, May 2026: DSA, version control, APIs, cloud basics, testing named as "the floor"; AI/ML, cybersecurity, DevOps expected by mid/senior; GenAI/Agentic AI as the high-pay track.

Curriculum area	R22 coverage	Gap found
Cloud (Azure/AWS/GCP)	Not in core — exists as elective	Partial — promote elective to core
Kubernetes / CI-CD	Manual kubectl, Jenkins	Confirms DevOps UPDATE
Git / version control	Never formally taught	Double-sourced — JD3 + grad interview
Microservices / REST APIs	Not covered	Full gap — explicit in JD2
AI/ML/GenAI	Covered, classical only	Confirms AI UPDATE
DSA / OOP / Algorithms	Strong, unchanged	No gap — validates KEEP
Databases	RDBMS-only	Mild — matches DBMS UPDATE

Section 6 — 2030 redesign proposal

Budget: 160 total credits, $\pm 20\text{--}25\%$ change cap $\rightarrow$ 32–40 credits may shift.

ADD – funded by Compiler Design merge savings (3cr freed)

- Git & Version Control (formal module) — 1cr — double-sourced gap
- Microservices & REST API Design — 2cr — explicit JD2 requirement

REDUCE

- Organizational Behaviour — keep R25's drop
- Open Electives — restrict to CS-only options

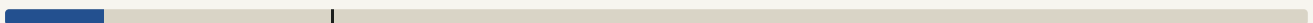
UPDATE – same credit slot, refreshed content

- Cloud Computing — promote from PE IV to core, refresh for multi-cloud/Terraform
- DevOps tooling — GitHub Actions, ArgoCD, Terraform, Prometheus
- AI core — add LLM/RAG/agents module
- DBMS — add NoSQL/vector DB module
- Cryptography — add cloud security/zero-trust module
- Computer Networks — VPCs/load balancers/edge, replacing RIP-only focus
- Compiler Design — modern tooling lens (ESLint/Babel-style)
- ML/Statistics — fix sequencing, move Stats to I-II/II-I

KEEP

- Data Structures & Algorithms
- Operating Systems
- Design & Analysis of Algorithms

Net credit shift against the cap



~3cr shifted (Git + Microservices, funded by the Compiler Design merge) — well inside the 32-40cr cap. Line marks the cap boundary.

AI capability test

Six subjects, one task each. The same pattern held every time: AI resolves syntax; it doesn't resolve judgment.

Subject	Task tested	Result
DSA	Balanced BST insert/delete	Solved instantly — doesn't replace tradeoff reasoning
DevOps	GitHub Actions CI/CD w/ Docker	Working YAML in one shot — confirms UPDATE toward architecture, not syntax
Machine Learning	Build & tune a classifier	Code yes, tuning judgment generic/unjustified
Compiler Design	Lexer/parser for a toy language	Works, struggles on grammar ambiguity — supports tooling-first UPDATE
Cryptography	Implement RSA key exchange	Solves textbook RSA — real PKI/system design still needs humans
Operating Systems	Explain scheduling tradeoffs for a workload	Generic correct answer, no context-specific reasoning — confirms KEEP

Human conversation

2024 CSE grad, now Platform Engineer, Hyderabad. Anonymous, ~20 minutes.

Most outdated course on day one: DevOps lab. College taught Jenkins freestyle jobs, docker run, Selenium IDE. First week: debugging a GitHub Actions YAML and an out-of-sync ArgoCD, having never opened a workflows file before.

Day-one skill the degree never taught: Git beyond commit/push — rebasing, merge conflicts, PR reviews. Force-pushed to main on day one; reverted by their lead.

More valuable in hindsight than it felt then: Operating Systems — semaphores and paging now map directly to pods getting OOMKilled or CPU-throttled in Kubernetes.

One credit they'd add for 2030: "Shipping with GenAI + Cloud" — practical RAG, vector DBs, prompt evaluation, cost control, deploying via Terraform, handling API keys and rate limits.

Did theory-heavy courses hold up? DAA — yes, directly useful for reasoning about production timeouts. Compiler Design's hand-written parser work — rarely used, but AST/optimization concepts still matter if taught through tools like Babel/ESLint.

Section 7 — What would make this wrong

- The ML/Stats sequencing flaw is wrong if R25 faculty confirm the co-placement was a deliberate just-in-time teaching choice.
- The DevOps staleness argument is wrong if placement data shows most grads don't enter cloud-native roles.
- The AI capability test's pattern is wrong if harder, multi-step agentic tasks show AI also handling judgment-based work.
- The 20-25% credit cap could be too conservative — but the human conversation points to tooling-level gaps, not whole missing domains, which suggests the cap holds.

Section 8 — AI claim checked, found wrong

CLAIM CHECKED

The HKAIVERSE brief's worked example states R25 cut DevOps from 4 credits to a 1-credit skill course.

METHOD

Cross-checked against two primary sources: the official JNTUH-hosted R25 bulletin PDF (accessed 15 Sept 2026) and the uploaded R25 syllabus PDF.

FINDING

Both sources show DevOps unchanged at 3+1 credits (4cr total) in III-I — the brief's claim does not hold. One caveat: confirm the exact brief PDF version before final submission.
